



FECHA ENTREGA: 16-9-2026

INSTITUTO TECNOLÓGICO SUPERIOR DE MISANTLA

MATERIA:

TALLER DE BASES DE DATOS

CATEDRÁTICO:

DR. IRAHÁN OTONIEL JOSE
GUZMÁN

SEMESTRE: 5°

GRUPO:503

UNIDAD: II

ACTIVIDAD:

REPORTE DE CREACIÓN DE
TABLAS, INSERCIÓN,
ACTUALIZACIÓN Y ELIMINACIÓN
DE DATOS.

ALUMNO:

SALOMÓN HERNÁNDEZ
CANDANEDO

Contenido

INTRODUCCIÓN	2
Desarrollo	3
CONCLUSIÓN	18

INTRODUCCIÓN

Las bases de datos permiten almacenar y organizar información de forma estructurada, facilitando su consulta y administración. Para trabajar con estos datos es necesario conocer las instrucciones que permiten crear las estructuras donde se almacenará la información y posteriormente realizar diferentes operaciones sobre ella.

En esta práctica se aborda el uso de instrucciones básicas del lenguaje SQL para la administración de datos. Para ello, se plantearon diferentes situaciones relacionadas con un restaurante, una escuela y una tiendita escolar, en las cuales es necesario almacenar información de manera organizada.

A lo largo de la práctica se trabaja con la creación de tablas y con las operaciones INSERT, UPDATE y DELETE, utilizando diferentes tipos de datos y restricciones como llaves primarias, campos obligatorios y valores predeterminados. El desarrollo de los ejercicios se realiza mediante SQLite y Python en Google Colab, permitiendo observar de manera práctica el funcionamiento de estas instrucciones dentro de una base de datos.

Desarrollo

Ejercicio — CREATE TABLE comida (Realizar un INSERT, UPDATE Y DELETE)

Una cadena de restaurantes necesita registrar en su base de datos el menú de platillos que ofrece. Crea una tabla llamada comida que cumpla con las siguientes especificaciones:

Script de SQLite en Colab:

```
import sqlite3

import pandas as pd

# Conexión en memoria (se pierde al cerrar el notebook)

conn = sqlite3.connect(":memory:")

cur = conn.cursor() #objeto para ejecutar comandos de SQL

def query(sql, params=()):

    """Ejecuta un SELECT y devuelve un DataFrame para verlo bonito."""

    return pd.read_sql_query(sql, conn, params=params)

def run(sql, params=()):

    """Ejecuta INSERT/UPDATE/DELETE/DDL y hace commit."""

    cur.execute(sql, params)

    conn.commit()

    print("OK ->", sql.strip().splitlines()[0], "...")
```

Creamos la tabla comida:

```
cur.executescript("""

DROP TABLE IF EXISTS comida;

CREATE TABLE comida (

    id_platillo INTEGER PRIMARY KEY AUTOINCREMENT,
```

```

nombre TEXT NOT NULL,

categoria TEXT,

precio REAL NOT NULL,

disponible BOOLEAN DEFAULT TRUE

);

""

conn.commit()

print("Tabla creada: comida")

```

#Insertamos unos datos a la tabla:

```

run("""INSERT INTO comida (nombre, categoria, precio) VALUES

('Ensalada César', 'Entrada', 85.00),

('Arroz con pollo', 'Plato fuerte', 150.00),

('Pasta Alfredo', 'Plato fuerte', 180.00),

('Pastel de chocolate', 'Postre', 75.00),

('Agua de horchata', 'Bebida', 35.00);""")

query("SELECT * FROM comida")

```

... OK -> INSERT INTO comida (nombre, categoria, precio) VALUES ...

	id_platillo	nombre	categoria	precio	disponible
0	1	Ensalada César	Entrada	85.0	1
1	2	Arroz con pollo	Plato fuerte	150.0	1
2	3	Pasta Alfredo	Plato fuerte	180.0	1
3	4	Pastel de chocolate	Postre	75.0	1
4	5	Agua de horchata	Bebida	35.0	1

1. Insertamos algunos datos a nuestra tabla y con query("SELECT * FROM comida") mostramos esta tabla con sus respectivos datos.

#Actualizamos precio a Pasta Alfredo de la tabla comida:

```
run("""  
UPDATE comida  
SET precio = 195.00  
WHERE nombre = 'Pasta Alfredo';  
""")  
query("SELECT * FROM comida")
```

OK -> UPDATE comida ...

	id_platillo	nombre	categoria	precio	disponible
0	1	Ensalada César	Entrada	85.0	1
1	2	Arroz con pollo	Plato fuerte	150.0	1
2	3	Pasta Alfredo	Plato fuerte	195.0	1
3	4	Pastel de chocolate	Postre	75.0	1
4	5	Agua de horchata	Bebida	35.0	1

2. En esta consulta actualizamos el precio de Pasta Alfredo y con query("SELECT * FROM comida") mostramos el resultado si se actualizo.

#Actualizamos el campo disponible a Agua de horchata de la tabla comida:

```
run("""  
UPDATE comida  
SET disponible = FALSE  
WHERE nombre = 'Agua de horchata';  
""")  
query("SELECT * FROM comida")
```

OK -> UPDATE comida ...

	id_platillo	nombre	categoria	precio	disponible
0	1	Ensalada César	Entrada	85.0	1
1	2	Arroz con pollo	Plato fuerte	150.0	1
2	3	Pasta Alfredo	Plato fuerte	195.0	1
3	4	Pastel de chocolate	Postre	75.0	1
4	5	Agua de horchata	Bebida	35.0	0

3. En esta consulta se cambia el valor de disponible a Agua de horchata y como este dato es de tipo booleano(Falso o Verdadero) en nuestra base de datos toma Verdadero = 1 y Falso = 0 y con la consulta query("SELECT * FROM comida") nos muestra el resultado del UPDATE.

#Eliminamos una fila de la tabla comida

```
run("""
DELETE FROM comida
WHERE nombre = 'Pastel de chocolate';
""")
query("SELECT * FROM comida")
```

*** OK -> DELETE FROM comida ...

	id_platillo	nombre	categoria	precio	disponible
0	1	Ensalada César	Entrada	85.0	1
1	2	Arroz con pollo	Plato fuerte	150.0	1
2	3	Pasta Alfredo	Plato fuerte	195.0	1
3	5	Agua de horchata	Bebida	35.0	0

4. En esta consulta eliminamos Pastel de chocolate de la tabla comida y con query("SELECT * FROM comida0") mostramos el resultado del DELETE.

Ejemplo 2 — Escuela (tabla alumnos) (Realizar CREATE, INSERT, UPDATE Y DELETE)

Una escuela quiere registrar la información básica de sus alumnos.

Script de SQLite en Colab:

```
import sqlite3

import pandas as pd

# Conexión en memoria

conn = sqlite3.connect(":memory:")

cur = conn.cursor()

def query(sql, params=()):

    """Ejecuta un SELECT y devuelve un DataFrame."""

    return pd.read_sql_query(sql, conn, params=params)

def run(sql, params=()):

    """Ejecuta INSERT/UPDATE/DELETE/DDI y hace commit."""

    cur.execute(sql, params)

    conn.commit()

    print("OK ->", sql.strip().splitlines()[0], "...")
```


CREAR TABLA alumnos:

```
cur.executescript("""
DROP TABLE IF EXISTS alumnos;

CREATE TABLE alumnos (
    id_alumno INTEGER PRIMARY KEY AUTOINCREMENT,
    nombre TEXT NOT NULL,
    grado TEXT,
    edad INTEGER NOT NULL,
    promedio REAL DEFAULT 0.0
);
""")

conn.commit()

print("Tabla creada: alumnos")
```

#Insertamos unos datos a la tabla alumnos

```
run("""INSERT INTO alumnos (nombre, grado, edad, promedio) VALUES
('Ana López', '3ro A', 8, 9.2),
('Luis Pérez', '5to B', 10, 8.7),
('María García', '4to A', 9, 9.5),
('Carlos Hernández', '6to B', 11, 8.9),
('Sofía Martínez', '3ro B', 8, 9.8);""")

query("SELECT * FROM alumnos")
```

... OK -> INSERT INTO alumnos (nombre, grado, edad, promedio) VALUES

	id_alumno	nombre	grado	edad	promedio
0	1	Ana López	3ro A	8	9.2
1	2	Luis Pérez	5to B	10	8.7
2	3	María García	4to A	9	9.5
3	4	Carlos Hernández	6to B	11	8.9
4	5	Sofía Martínez	3ro B	8	9.8

5. Mostramos los datos que se insertamos en nuestra tabla alumnos.

#Hacemos una segunda inserción:

```
run("""INSERT INTO alumnos (nombre, grado, edad) VALUES
```

```
('Jorge Ramírez', '5to A', 10);""")
```

```
query("SELECT * FROM alumnos")
```

OK -> INSERT INTO alumnos (nombre, grado, edad) VALUES ...

	id_alumno	nombre	grado	edad	promedio
0	1	Ana López	3ro A	8	9.2
1	2	Luis Pérez	5to B	10	8.7
2	3	María García	4to A	9	9.5
3	4	Carlos Hernández	6to B	11	8.9
4	5	Sofía Martínez	3ro B	8	9.8
5	6	Jorge Ramírez	5to A	10	0.0

6. En esta segunda inserción no pusimos id_alumno ni promedio porque ya están definidos en nuestra base de datos el id_alumnos es auto_increment y el promedio es por default = 0.0.

#Actualizamos el promedio a Luis Pérez:

```
run("""
```

```
UPDATE alumnos
```

```
SET promedio = 9.1
```

```
WHERE nombre = 'Luis Pérez';
```

```
""")
```

```
query("SELECT * FROM alumnos")
```

OK -> UPDATE alumnos ...

	id_alumno	nombre	grado	edad	promedio
0	1	Ana López	3ro A	8	9.2
1	2	Luis Pérez	5to B	10	9.1
2	3	María García	4to A	9	9.5
3	4	Carlos Hernández	6to B	11	8.9
4	5	Sofía Martínez	3ro B	8	9.8
5	6	Jorge Ramírez	5to A	10	0.0

7. En esta consulta actualizamos el promedio de Luis Pérez y los mostramos con query("SELECT * FROM alumnos").

#Actualizamos el grado a Carlos Hernandez:

```
run("""
```

```
UPDATE alumnos
```

```
SET grado = '6to A'
```

```
WHERE nombre = 'Carlos Hernández';
```

```
""")
```

```
query("SELECT * FROM alumnos")
```

OK -> UPDATE alumnos ...

	id_alumno	nombre	grado	edad	promedio
0	1	Ana López	3ro A	8	9.2
1	2	Luis Pérez	5to B	10	9.1
2	3	María García	4to A	9	9.5
3	4	Carlos Hernández	6to A	11	8.9
4	5	Sofía Martínez	3ro B	8	9.8
5	6	Jorge Ramírez	5to A	10	0.0

8. En esta consulta cambiamos de 5toB a 6toA a Luis Pérez.

#Hacemos un Delete:

```
run("""
DELETE FROM alumnos
WHERE nombre = 'Jorge Ramírez';
""")
query("SELECT * FROM alumnos")
```

OK -> DELETE FROM alumnos ...

	id_alumno	nombre	grado	edad	promedio
0	1	Ana López	3ro A	8	9.2
1	2	Luis Pérez	5to B	10	9.1
2	3	María García	4to A	9	9.5
3	4	Carlos Hernández	6to A	11	8.9
4	5	Sofía Martínez	3ro B	8	9.8

9. Y por último hacemos una consulta DELETE donde de eliminamos a Jorge Ramírez de la tabla alumnos y con un SELECT * FROM alumnos mostramos el resultado final de nuestra tabla.

Ejemplo 3 — Tiendita escolar (tabla productos) (Realizar CREATE, INSERT, UPDATE Y DELETE)

La tiendita de la escuela necesita llevar el control de los productos que vende.

```
import sqlite3

import pandas as pd

# Conexión en memoria

conn = sqlite3.connect(":memory:")

cur = conn.cursor()

def query(sql, params=()):

    """Ejecuta un SELECT y devuelve un DataFrame."""

    return pd.read_sql_query(sql, conn, params=params)


def run(sql, params=()):

    """Ejecuta INSERT/UPDATE/DELETE/DDL y hace commit."""

    cur.execute(sql, params)

    conn.commit()

    print("OK ->", sql.strip().splitlines()[0], "...")
```

Creamos la tabla productos:

```
cur.executescript("""

DROP TABLE IF EXISTS productos;

CREATE TABLE productos (

    id_producto INTEGER PRIMARY KEY AUTOINCREMENT,

    nombre TEXT NOT NULL,
```

```

tipo TEXT,

precio REAL NOT NULL,

existencia INTEGER DEFAULT 0

);

""

conn.commit()

print("Tabla creada: productos")

```

#Hacemos un insert:

```

run("""INSERT INTO productos (nombre, tipo, precio, existencia) VALUES

('Sabritas', 'Snack', 18.00, 20),

('Coca-Cola', 'Bebida', 22.00, 15),

('Chocolate', 'Dulce', 15.00, 25),

('Paleta', 'Dulce', 8.00, 30),

('Cuaderno profesional', 'Papelería', 45.00, 10);""")

query("SELECT * FROM productos")

```

OK -> INSERT INTO productos (nombre, tipo, precio, existencia) VALUES .

	id_producto	nombre	tipo	precio	existencia
0	1	Sabritas	Snack	18.0	20
1	2	Coca-Cola	Bebida	22.0	15
2	3	Chocolate	Dulce	15.0	25
3	4	Paleta	Dulce	8.0	30
4	5	Cuaderno profesional	Papelería	45.0	10

10. Insertamos algunos datos a nuestra tabla productos.

#Hacemos una segunda inserción:

```
run("""INSERT INTO productos (nombre, tipo, precio) VALUES
```

```
('Goma', 'Papelería', 6.00);""")
```

```
query("SELECT * FROM productos")
```

OK -> INSERT INTO productos (nombre, tipo, precio) VALUES ...

	id_producto	nombre	tipo	precio	existencia
0	1	Sabritas	Snack	18.0	20
1	2	Coca-Cola	Bebida	22.0	15
2	3	Chocolate	Dulce	15.0	25
3	4	Paleta	Dulce	8.0	30
4	5	Cuaderno profesional	Papelería	45.0	10
5	6	Goma	Papelería	6.0	0

11. En esta segunda inserción id_producto y existencia ya están definidos en la base de datos por eso no los pusimos, y el id_producto se autoincrementa y la existencia por default se pone en 0 así que si no los ponemos no hay problema a la hora de ejecutar la instrucción y se hace la inserción sin problema.

#Actualizamos precio a Sabritas:

```
run("""
```

```
UPDATE productos
```

```
SET precio = 20.00
```

```
WHERE nombre = 'Sabritas';
```

```
""")
```

```
query("SELECT * FROM productos")
```

OK -> UPDATE productos ...

	id_producto	nombre	tipo	precio	existencia
0	1	Sabritas	Snack	20.0	20
1	2	Coca-Cola	Bebida	22.0	15
2	3	Chocolate	Dulce	15.0	25
3	4	Paleta	Dulce	8.0	30
4	5	Cuaderno profesional	Papelería	45.0	10
5	6	Goma	Papelería	6.0	0

12. Aquí utilizamos un UPDATE ósea que le cambiamos el precio de Sabritas antes costaba 18.0 y lo actualizamos a 20.0.

#Usamos otro UPDATE para actualizar la existencia a Coca-Cola:

```
run("""
UPDATE productos
SET existencia = 35
WHERE nombre = 'Coca-Cola';
""")
query("SELECT * FROM productos")
```


OK -> UPDATE productos ...

	id_producto	nombre	tipo	precio	existencia
0	1	Sabritas	Snack	20.0	20
1	2	Coca-Cola	Bebida	22.0	35
2	3	Chocolate	Dulce	15.0	25
3	4	Paleta	Dulce	8.0	30
4	5	Cuaderno profesional	Papelería	45.0	10
5	6	Goma	Papelería	6.0	0

13. En esta nueva consulta actualizamos la existencia al producto Coca-Cola antes tenia existencia = 15 y ahora tiene existencia = 35.

#Por último realizamos un DELETE:

```
run("""
DELETE FROM productos
WHERE nombre = 'Goma';
""")
query("SELECT * FROM productos")
```

OK -> DELETE FROM productos ...

	id_producto	nombre	tipo	precio	existencia
0	1	Sabritas	Snack	20.0	20
1	2	Coca-Cola	Bebida	22.0	35
2	3	Chocolate	Dulce	15.0	25
3	4	Paleta	Dulce	8.0	30
4	5	Cuaderno profesional	Papelería	45.0	10

14. En esta parte hacemos un DELETE en donde borramos el producto llamado Goma de la tabla productos y mostramos el resultado final de nuestra tabla para verificar si realmente se eliminó dicho producto.

CONCLUSIÓN

A través de los tres ejercicios realizados se pudo aplicar de manera práctica el manejo básico de una base de datos mediante SQLite y Python en Google Colab. En el primer ejercicio se creó la tabla comida, donde se registró información relacionada con platillos, categorías, precios y disponibilidad. Posteriormente, en el ejercicio de la tabla alumnos, se trabajó con datos como nombre, grado, edad y promedio, utilizando también un valor predeterminado para el promedio. Finalmente, en la tabla productos se llevó el control de artículos de una tiendita escolar mediante datos como nombre, tipo, precio y existencia.

En los tres casos se aplicaron las instrucciones CREATE TABLE, INSERT, UPDATE y DELETE, lo que permitió observar cómo se construye una tabla y cómo se pueden agregar, modificar y eliminar registros. También se utilizaron restricciones como la llave primaria autoincremental, campos que no pueden ser nulos y valores por defecto, comprobando su función dentro de cada tabla.

En conjunto, los ejercicios permitieron comprender que, aunque la información de cada situación es diferente, el procedimiento para organizarla y administrarla mediante SQL mantiene una estructura similar. El uso de ejemplos relacionados con un restaurante, una escuela y una tiendita escolar facilitó la comprensión de cómo las bases de datos pueden adaptarse a diferentes necesidades y situaciones reales.